

Birzeit University
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering

ENCS4320 – Applied Cryptography (Term 1252)

Final Course Project

SecureChannel

A Secure Communication Application

Wadee Owaise – 1230482
Hakam Snoubar – 1230152

Dr. Ahmad Shawahna

June 2026

*A from-scratch implementation of X25519, HMAC, HKDF, and
ChaCha20-Poly1305 AEAD in pure Python.*

Abstract

This report presents **SecureChannel**, a secure chat application built in pure Python for the ENCS4320 final project. Two parties (a client and a server) exchange messages over TCP with full confidentiality, integrity, and replay protection. The handshake uses X25519 to agree on a shared secret, HMAC with a pre-shared key (PSK) to authenticate both sides, and HKDF to derive session keys. All messages are then encrypted with ChaCha20-Poly1305 AEAD. Every primitive – SHA-256, HMAC, HKDF, ChaCha20, Poly1305, the AEAD construction, and X25519 – was written from scratch in pure Python and tested against the official RFC test vectors.

Contents

Abstract	1
1 Introduction	4
1.1 Project Overview	4
2 System Architecture	4
2.1 Two-Party Model	4
2.2 Phase A: Handshake	4
2.3 Phase B: Secure Messaging	4
3 Cryptographic Primitives	5
3.1 SHA-256	5
3.2 HMAC	5
3.3 HKDF	5
3.4 ChaCha20	6
3.5 Poly1305	6
3.6 ChaCha20-Poly1305 AEAD	6
3.7 X25519	6
4 Protocol Design – Handshake and Authentication	7
4.1 Handshake Message Format	7
4.2 Mutual Authentication via HMAC	7
4.3 Key Schedule	7
4.4 Reflection Attack Defense	8
5 Secure Channel – AEAD Messaging	8
5.1 SecureChannel State	8
5.2 Nonce Construction	8
5.3 Frame Format and Associated Data	9
5.4 Replay Protection	9
5.5 Tamper Detection and Message Types	9
6 Implementation	10
6.1 Server and Client Structure	10
6.2 Threading Model	10
7 Testing and Validation	10
7.1 Validation Strategy	10
8 Security Analysis	11
8.1 Confidentiality	11
8.2 Integrity	11
8.3 Authenticity and Man-in-the-Middle Defense	11
8.4 Replay and Reflection Defense	12
8.5 Known Limitations	12

9	AI Usage Statement	12
9.1	How the Tools Were Used	12
10	Individual Contribution Statement	12
10.1	Wadee Owaise – 1230482	12
10.2	Hakam Snoubar – 1230152	13
11	Conclusion	13
List of Tables		
1	Cryptographic components and their roles.	5

1 Introduction

1.1 Project Overview

SecureChannel is a two-party chat application. A client and a server connect over TCP and exchange messages with confidentiality, integrity, and replay protection. The same primitives used here – X25519, HKDF, and ChaCha20-Poly1305 – power real-world systems like TLS 1.3, Signal, and WireGuard, so the project is a small-scale version of what secures the modern internet. No crypto libraries are used inside the implementation.

2 System Architecture

2.1 Two-Party Model

SecureChannel has two programs: a client and a server. They communicate over a TCP socket. The two roles are almost identical cryptographically – both do the same X25519, HMAC, and HKDF operations. They differ only in who opens the TCP connection and in a role label inside the authentication tag.

The two parties share a long-term pre-shared key (PSK), set up out of band (for example, read from a local config file). The PSK is like a strong password the two users agreed on in person. It is never sent over the network. It is used to authenticate the handshake and as the salt for HKDF.

2.2 Phase A: Handshake

The handshake has three steps:

1. Each side generates an ephemeral X25519 key pair and they exchange public keys. Both compute the same shared secret.
2. Each side computes an HMAC tag over the handshake transcript (version, both identities, both public keys), keyed by the PSK, and sends it for verification. A bad tag aborts the connection.
3. The shared secret is fed into HKDF, with the PSK as salt, to derive two directional session keys and two initial nonces.

At the end of Phase A, both parties hold the same keys and nonces and have verified each other's identity.

2.3 Phase B: Secure Messaging

After the handshake, all messages are protected with ChaCha20-Poly1305 AEAD. Each message has a plaintext header (version, message type, sender length, sender identity, sequence number) that is authenticated as associated data (AAD) but not encrypted. The header is followed by the ciphertext length, the ciphertext, and a 16-byte Poly1305 tag.

The per-message nonce is derived by XOR-ing the initial nonce with the current sequence number. The receiver only accepts a frame whose sequence number matches its expected counter, which provides replay and reordering protection.

Table 1 summarizes the cryptographic components used in SecureChannel, their specifications, and their roles.

Table 1. Cryptographic components and their roles.

Component	Specification	Role in SecureChannel
SHA-256	RFC 4634	Hash used inside HMAC and HKDF.
HMAC	RFC 2104	Authenticates the handshake transcript.
HKDF	RFC 5869	Derives two session keys and two nonces from the shared secret and PSK.
ChaCha20	RFC 8439	Stream cipher that encrypts message bodies.
Poly1305	RFC 8439	One-time MAC that authenticates AAD, ciphertext, and lengths.
ChaCha20-Poly1305	RFC 8439	AEAD combining ChaCha20 and Poly1305.
X25519	RFC 7748	Ephemeral key exchange producing the shared secret.

3 Cryptographic Primitives

3.1 SHA-256

SHA-256 takes an arbitrary-length message and produces a 32-byte digest. The implementation pads the message to a multiple of 64 bytes, initializes eight 32-bit working variables to fixed constants, and runs 64 rounds of a compression function per block. SHA-256 is the foundation of HMAC and HKDF, so any bug here breaks everything above.

3.2 HMAC

HMAC builds a message authentication code from any hash function. Given a key K and message m :

$$\text{HMAC}_K(m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m)),$$

where $\text{ipad} = 0 \times 36$ repeated to 64 bytes and $\text{opad} = 0 \times 5c$ repeated to 64 bytes. The size 64 bytes matches the block size of SHA-256 (the underlying hash). HMAC works by processing data in hash-sized blocks: the key is first padded to the block size (or hashed first and then padded if it is longer than the block), then XOR-ed with ipad or opad to produce two different padding patterns. This ensures the inner and outer hash computations use distinct keyed prefixes, which is essential for the security proof of HMAC. Our implementation uses our own SHA-256 as the underlying hash. HMAC is used both directly (to authenticate the handshake) and inside HKDF.

3.3 HKDF

HKDF derives one or more keys from input keying material. It has two stages. The *extract* stage turns the (possibly biased) input into a pseudorandom key:

$$\text{PRK} = \text{HMAC}_{\text{salt}}(\text{IKM}),$$

where the salt is the PSK and the IKM is the X25519 shared secret. The *expand* stage then produces L bytes of output by chaining HMAC calls:

$$T(0) = \emptyset, \quad T(i) = \text{HMAC}_{\text{PRK}}(T(i-1) \parallel \text{info} \parallel \text{byte}(i)), \quad \text{OKM} = T(1) \parallel T(2) \parallel \dots$$

truncated to length L . The `info` field binds the derived key to a specific purpose; in SecureChannel it is `PROTOCOL_VERSION + b"PhaseA"`, so keys cannot be reused across versions or phases.

3.4 ChaCha20

ChaCha20 is a stream that takes a 32-byte key and a 12-byte nonce, and produces a keystream that is XOR-ed with the plaintext. The cipher maintains a 512-bit state arranged as a 4×4 matrix of 32-bit words: four fixed constants, eight key words, one counter word, and three nonce words. To produce each 64-byte keystream block, the cipher applies 20 rounds (organized as 10 double-rounds) to the state.

The core operation is the *quarter-round*, which takes four 32-bit words (a, b, c, d) and performs a sequence of add–XOR–rotate operations: it adds two words together (modulo 2^{32}), XORs the result into a third word, and rotates that word by a fixed number of bits. This sequence is repeated four times per quarter-round with different word assignments and rotation amounts (16, 12, 8, and 7 bits). Each double-round consists of four *column rounds* (operating on the four columns of the state matrix) followed by four *diagonal rounds* (operating on the four diagonals). After all 20 rounds, the original state is added back to the working state word-by-word, and the result is serialized in little-endian byte order to produce the 64-byte keystream block. The counter is incremented for each subsequent block so that each block produces a different keystream.

3.5 Poly1305

Poly1305 is a one-time MAC based on polynomial evaluation over the prime field $\mathbb{F}_p$ with $p = 2^{130} - 5$. The 32-byte key is split into a 16-byte value r (with certain bits clamped) and a 16-byte value s . For a message split into 16-byte blocks $m_1, \dots, m_q$, the tag is

$$\text{tag} = \left(\sum_{i=1}^q (\text{int}(m_i \parallel 0 \times 01) \cdot r^{q-i+1}) \bmod p \right) + s \pmod{2^{128}}.$$

Because Poly1305 keys are one-time, we derive a fresh key for every message from the first 32 bytes of ChaCha20 block 0, exactly as specified in RFC 8439.

3.6 ChaCha20-Poly1305 AEAD

The AEAD construction defined in RFC 8439 combines ChaCha20 (for encryption) and Poly1305 (for authentication). Given a key, a 12-byte nonce, plaintext, and optional associated data (AAD), the construction works in three steps. First, it derives a one-time Poly1305 key from the first 32 bytes of ChaCha20 block 0 (using counter value 0). Second, it encrypts the plaintext with ChaCha20 starting at counter 1, producing the ciphertext. Third, it computes the Poly1305 tag over a structured input: the AAD padded to a multiple of 16 bytes, followed by the ciphertext padded to a multiple of 16 bytes, followed by the 8-byte little-endian length of the AAD and the 8-byte little-endian length of the ciphertext.

Decryption reverses the process: it recomputes the Poly1305 tag over the received AAD, ciphertext, and lengths, and rejects the message if the recomputed tag does not match the received tag. This prevents any tampering with either the header or the body – a single flipped bit anywhere causes the tag verification to fail.

3.7 X25519

X25519, defined in RFC 7748, is an elliptic-curve Diffie–Hellman key exchange over the curve $y^2 = x^3 + 486662x^2 + x$ in the field $\mathbb{F}_{2^{255}-19}$. Each party generates a 32-byte private key, computes the public key as a scalar multiplication of the base point $u = 9$, and exchanges public keys. The shared secret is the scalar multiplication of the peer’s public key by one’s own private key.

The implementation uses the Montgomery ladder, a constant-time scalar multiplication algorithm that avoids branch-dependent timing leaks. The ladder processes the private scalar bit

by bit, maintaining two points whose difference is the input point u , and performs a differential add-and-double step at each bit. A branchless XOR-based conditional swap keeps the operation constant-time.

4 Protocol Design – Handshake and Authentication

4.1 Handshake Message Format

The handshake exchanges two messages of identical structure, one in each direction. The client, having generated an ephemeral X25519 key pair (b, B) , sends the following to the server:

```
[public key B, 32 bytes] [version, 2 bytes] [client_id length, 1 byte] [client_id, N bytes]
```

The server checks that the version matches, generates its own ephemeral key pair (a, A) , and replies with the same format using its own public key and identity:

```
[public key A, 32 bytes] [version, 2 bytes] [server_id length, 1 byte] [server_id, N bytes]
```

Both parties then compute the shared secret $DH = a \cdot B = b \cdot A$ using X25519. If the protocol versions do not match, the connection is closed immediately.

4.2 Mutual Authentication via HMAC

After the ephemeral keys are exchanged, both parties authenticate by computing an HMAC tag keyed by the PSK over a transcript that includes the version, both identities, and both ephemeral public keys. The tag also includes a role label – b"client_auth" or b"server_auth" – that distinguishes the direction. The client sends:

$$\text{tag}_c = \text{HMAC}_{\text{PSK}}(\text{b"client_auth"} \parallel \text{version} \parallel \text{server_id} \parallel \text{client_id} \parallel A \parallel B)$$

and the server verifies it. The server then sends:

$$\text{tag}_s = \text{HMAC}_{\text{PSK}}(\text{b"server_auth"} \parallel \text{version} \parallel \text{server_id} \parallel \text{client_id} \parallel A \parallel B)$$

and the client verifies it. A few design points are worth noting. First, the two MACs are *different* – they use different role labels – so a server's tag cannot be reflected back to fool the client and vice versa. Second, the transcript is identical on both sides: same version, same identities in the fixed order `server_id`, `client_id`, same public keys in the fixed order A (server), B (client). The order is protocol-defined, so there is no ambiguity. Third, the role labels are also protocol-defined: every implementation uses exactly the strings `b"client_auth"` and `b"server_auth"`.

4.3 Key Schedule

After authentication, both parties derive session key material using HKDF. Inputs: PSK as salt, X25519 shared secret as IKM, `version + b"PhaseA"` as info. Output: 88 bytes, split into four pieces (Listing 1).

```
1 mixer = HKDF()
2 key=mixer.hkdf(psk,shared_secret,version + b" PhaseA", 88)
3
4 key_server_to_client = key[:32]      # server -> client encryption key
5 key_client_to_server = key[32:64]    # client -> server encryption key
6 nonce_server_to_client = key[64:76]  # 12-byte initial nonce (s -> c)
7 nonce_client_to_server = key[76:]    # 12-byte initial nonce (c -> s)
```

Listing 1. HKDF key schedule.

Each party constructs a `SecureChannel` with these values in the order matching its role: the server uses `key_server_to_client` as its send key and `key_client_to_server` as its receive key; the client does the opposite. This directional separation means the two sides never encrypt with the same key, which prevents a class of reflection attacks against the AEAD layer.

4.4 Reflection Attack Defense

A reflection attack is when an attacker captures a party's authentication tag and sends it back to that same party, hoping to fool it into accepting the tag as proof of identity from the peer. The defense is simple: the two directions of authentication use *different* role labels (`b"client_auth"` and `b"server_auth"`), so a tag computed by the client can never be valid as a server authentication tag and vice versa.

This answers a design question from the original draft: *should the MAC sent from server to client be the same as the one sent from client to server?* No – they must differ, and they differ through the role label. The transcript bytes (everything except the label) are the same on both sides, which is essential: if they differed, an attacker could potentially substitute one party's view for the other's.

5 Secure Channel – AEAD Messaging

5.1 SecureChannel State

After the handshake, both parties construct a `SecureChannel` object holding the session state: a socket, a version, a send cipher, a send nonce, a send sequence counter, a receive cipher, a receive nonce, a receive sequence counter, the sender identity, and a `closed` flag. The send and receive ciphers are separate `ChaCha20Poly1305` instances keyed with the appropriate directional key, so each direction has its own independent key and nonce evolution. Listing 2 shows the initialization.

```
1 class SecureChannel:
2     def __init__(self, sock, send_key, recv_key,
3                 send_nonce, recv_nonce, version, sender):
4         self.sock = sock
5         self.version = version
6         # Sending state
7         self.send_cipher = ChaCha20Poly1305(send_key)
8         self.send_nonce = send_nonce          # 12 bytes from HKDF
9         self.send_seq = 0
10        # Receiving state
11        self.recv_cipher = ChaCha20Poly1305(recv_key)
12        self.recv_nonce = recv_nonce          # 12 bytes from HKDF
13        self.recv_seq = 0
14        self.sender = sender
15        self.closed = False
```

Listing 2. SecureChannel state.

5.2 Nonce Construction

The per-message nonce is built by XOR-ing the 12-byte initial nonce (from HKDF) with the current sequence number encoded as a 12-byte little-endian integer (Listing 3). This gives us three things: uniqueness (the sequence number strictly increases and the initial nonces differ by direction), statelessness (the nonce can be recomputed at any time from the initial nonce and

the current sequence number), and compatibility with RFC 8439’s recommendation to derive the nonce from a counter and a base value (the same approach TLS 1.3 uses).

```
1 def _nonce_for_seq(self, initial_nonce, seq_num):
2     seq_bytes = struct.pack('<Q', seq_num).rjust(12, b'\x00')
3     return bytes(a ^ b for a, b in zip(initial_nonce, seq_bytes))
```

Listing 3. Per-message nonce derivation.

The single most important rule of ChaCha20-Poly1305 is that the (key, nonce) pair must never be reused. If it is, confidentiality is broken (the same keystream is XOR-ed with two plaintexts, revealing their XOR) and integrity is broken (Poly1305 keys derived from the same keystream block can be combined to forge tags). The strict-increase discipline on `send_seq`, together with the directional separation of keys and the per-direction initial nonces, ensures the (key, nonce) pair is fresh for every message.

5.3 Frame Format and Associated Data

Each frame begins with a plaintext header authenticated as AAD but not encrypted. The AAD header has the following format:

```
[version, 2B] [msg_type, 1B] [seq, 1B] [sender_len, 1B] [sender, NB]
```

The complete frame on the wire is:

```
[AAD header] [ciphertext_len, 4B] [ciphertext, NB] [tag, 16B]
```

The ciphertext length is stored as a 4-byte big-endian integer. Because the header is part of the AAD, any modification to the version, message type, sequence number, or sender identity causes the Poly1305 tag verification to fail. An attacker cannot, for example, change the message type from 1 (text) to 2 (close) without invalidating the tag.

5.4 Replay Protection

The receiver enforces strict ordering: it accepts a frame only if its sequence number equals the receiver’s expected `recv_seq`, and it increments the counter after each successful decryption. If an attacker captures a frame and replays it, the replayed frame’s sequence number will be less than `recv_seq`, and the receiver will tear down the channel. The same mechanism rejects reordered frames.

The sequence number is a single byte, so a session can hold at most 256 messages per direction before the counter wraps. This is fine for a course project; a production system would use a larger counter (32 or 64 bits) or perform a key rotation before the wrap.

5.5 Tamper Detection and Message Types

Tamper detection is provided by the Poly1305 tag, which covers the AAD, the ciphertext, and the lengths. Any modification – a flipped bit in the header, a changed byte in the ciphertext, or a change to a length field – causes the recomputed tag to differ from the received tag, and the `decrypt` method raises an exception. The receive loop catches this exception, prints a diagnostic, and closes the connection.

The protocol defines two message types: type 1 is a normal text message (the plaintext is the UTF-8 encoding of the user’s input), and type 2 is a close request with an empty plaintext. The user types `:eq` to send a type-2 frame and exit.

6 Implementation

6.1 Server and Client Structure

The server and client share almost all their code. They differ only in who opens the TCP connection, in the order of the handshake messages, and in the role label used in the authentication tag. The server creates a listening TCP socket, accepts a connection, parses the client's hello message, generates its own ephemeral key pair, sends its hello message, computes the shared secret, runs the authentication exchange, and constructs a `SecureChannel`. The client opens a TCP socket to the server, generates its ephemeral key pair, sends its hello message, parses the server's hello, computes the shared secret, runs the authentication exchange, and constructs a `SecureChannel`. After the handshake, both sides enter the same main loop: read user input, send it as a type-1 message, and check for the `:eq` command to close.

6.2 Threading Model

To enable simultaneous send and receive, each program runs two threads after the handshake. The main thread blocks on `input()` waiting for the user to type a message; when the user presses Enter, the main thread sends the message and loops. A background worker thread, started before the input loop, blocks on `channel.receive_message` and prints each received message prefixed with `HIM:`. When the worker receives a type-2 frame, it prints a closing message and close down the channel.

The worker thread (Listing 4) catches `ConnectionAbortedError`, `ConnectionResetError`, and `OSError` (covering the cases where the socket is closed by the peer or by the main thread). It also catches generic `Exception` as a last resort, to ensure that a tampered or malformed frame does not cause an exception (If the secure channel object raised an exception)

```
1 def worker(channel: SecureChannel):
2     while not channel.closed:
3         try:
4             msg_type, plaintext = channel.receive_message()
5             print("HIM: " + plaintext.decode())
6             if msg_type == 2:
7                 print("Closing Connection...")
8                 channel.close()
9                 return
10        except (ConnectionAbortedError, ConnectionResetError, OSError):
11            print("\n[!] Connection lost.")
12            channel.close()
13            return
14        except Exception as e:
15            print(f"\n[!] Unexpected error: {e}")
16            channel.close()
17            return
```

Listing 4. Worker thread: receive loop.

7 Testing and Validation

7.1 Validation Strategy

Every primitive was validated against the official test vectors published in its RFC specification. The team wrote a single test file (`validator.py`) that imports the team's own modules and

runs each primitive on the official test inputs, comparing the output byte-for-byte against the expected output from the RFC. No external cryptographic libraries were used at all – only the team’s own implementations were tested against the published test vectors.

The test file imports the modules and defines a separate validation function for each primitive. Each function loads the official test vectors taken from the relevant RFC, runs the implementation, and prints PASS or FAIL. All tests pass, confirming that every primitive matches its specification. Listing 5 shows the complete test output.

```
Validating ChaCha20-Poly1305 AEAD
PASS: b'Ladies and'...
Validating ChaCha20
PASS: b'Ladies and'...
Validating Poly1305
PASS: b'Cryptograp'...
Validating SHA256
PASS: b''...
PASS: b'abc'...
PASS: b'abcbcbcdcd'...
PASS: b'aaaaaaaaaa'...
Validating HMAC
PASS: b'Hi There'...
PASS: b'what do ya'...
PASS: b'\xdd\xdd\xdd\xdd\xdd\xdd\xdd\xdd\xdd'...
PASS: b'\xcd\xcd\xcd\xcd\xcd\xcd\xcd\xcd\xcd'...
PASS: b'Test With '...
PASS: b'Test Using'...
PASS: b'This is a '...
Validating HDKF
PASS: b'\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9'...
PASS: b'\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9'...
PASS: b''...
Validating X25519
PASS: b'w\x07m\ns\x18\xa5}<\x16'...
PASS: b'j\xab\x08~bJ\x8aKy\xel'...
```

Listing 5. Test output from `validator.py` (all tests pass).

8 Security Analysis

8.1 Confidentiality

Confidentiality comes from ChaCha20, which produces a fresh keystream keyed by the session key and the per-message nonce. Because the (key, nonce) pair is unique per message, the keystream is never reused, and the XOR of two ciphertexts reveals nothing about the plaintexts. An attacker observing the wire learns nothing about message contents beyond their lengths.

8.2 Integrity

Integrity comes from the Poly1305 one-time MAC, which covers the AAD, the ciphertext, and the lengths. The Poly1305 key is fresh for every message. Forging a tag without the key requires solving a discrete logarithm in $\mathbb{F}_{2^{130}-5}$ or guessing the 128-bit tag, both infeasible. Any modification causes the recomputed tag to differ, and the message is rejected.

8.3 Authenticity and Man-in-the-Middle Defense

Handshake authenticity comes from the HMAC transcript tag keyed by the PSK. Because the PSK is known only to the two legitimate parties, an attacker cannot compute a valid tag for a transcript they did not produce. The transcript binds together the version, both identities, and both ephemeral public keys, so an attacker cannot substitute their own ephemeral key – doing so would change the transcript and invalidate the tag. This is the central defense against

man-in-the-middle attacks: an attacker who substitutes their own keys for both directions cannot produce HMAC tags that the legitimate parties will accept, because they do not know the PSK.

8.4 Replay and Reflection Defense

Replay is detected by the strict receive-sequence check. A replayed frame carries a sequence number less than the receiver's expected counter, so it is rejected. Reflection is detected by the role label in the HMAC tag: a tag computed by the client uses `b"client_auth"` and is verified by the server using the same label; a tag computed by the server uses `b"server_auth"` and is verified by the client. An attacker who reflects a client's tag back to the client will be detected, because the client expects the `b"server_auth"` label and finds a mismatch.

8.5 Known Limitations

The system has two known limitations worth noting. First, the PSK must be distributed out of band; if it is compromised, an attacker can act as either party in future sessions. Second, the 8-bit sequence number limits a session to 256 messages per direction before wrap-around.

9 AI Usage Statement

During the project the team used general-purpose AI assistants (Gemini and Claude) via their web interfaces.

9.1 How the Tools Were Used

The team used the AI assistants strictly within the permitted uses listed in Section 7 of the project specification:

- **Debugging and syntax lookup.** When an implementation produced an unexpected error or a Python syntax question arose (for example, the difference between `bytearray` and `bytes` for in-place XOR), the team consulted the AI assistant for an explanation and a fix. Every recommendation was verified against the official Python documentation before being applied.
- **Grammar and clarity feedback.** An early draft of this report and the project README were reviewed by the AI assistant for grammar and clarity. The team meaningfully edited every suggestion.
- **Conceptual explanations.** The team used the AI assistant as a tutor to ask conceptual questions, such as why Poly1305 keys must be one-time.
- **Boilerplate and utility code.** The AI assistant suggested the structure of the socket setup, the threading boilerplate, and the command-line argument parsing. These are utility components explicitly permitted under Section 7. The team read, understood, and adapted every suggestion.

10 Individual Contribution Statement

Documentation (Report & README) was a joint effort. The development and coding workload was divided as follows:

10.1 Wadee Owaise – 1230482

I implemented SHA-256, HMAC, and HKDF, including their test vectors from RFC 4634, RFC 4231, and RFC 5869 respectively. I also created the server-client communication channel, including the `SecureChannel` class, the frame format, the nonce derivation, and the replay-protection sequence check. I wrote the server and client main loops and the threading model that enables simultaneous send and receive.

10.2 Hakam Snoubar – 1230152

I implemented ChaCha20, Poly1305, the ChaCha20-Poly1305 AEAD construction, and X25519, including their test vectors from RFC 8439 and RFC 7748. I wrote the Montgomery ladder for X25519 scalar multiplication, the ChaCha20 quarter-round and block function, and the Poly1305 polynomial evaluation. I also contributed to the security analysis section of this report.

***Note on Commit History:** Because we occasionally shared files directly to avoid merge conflicts, Git commit counts do not accurately reflect our actual division of programming labor.*

11 Conclusion

The project tied together symmetric-key cryptography (SHA-256, HMAC, ChaCha20, Poly1305), asymmetric-key cryptography (X25519), key derivation (HKDF), and authenticated encryption (ChaCha20-Poly1305) into a single system that resists replay, reflection, tampering, and man-in-the-middle attacks. Validating every primitive against its official test vectors caught several subtle bugs during development and gave the team confidence that the code matches the specifications.

The main lessons learned: composition matters as much as the primitives themselves (a correct X25519 is useless if the handshake does not bind the ephemeral keys to identities; a correct ChaCha20-Poly1305 is useless if the (key, nonce) pair is ever reused); every design decision has a corresponding attack it defends against (role label → reflection, sequence counter → replay, AAD → header tampering, PSK → man-in-the-middle); and reading specifications carefully is a core skill.

References

- [1] D. Eastlake 3rd, T. Hansen. *US Secure Hash Algorithms (SHA and HMAC-SHA)*. RFC 4634, July 2006.
- [2] H. Krawczyk, M. Bellare, R. Canetti. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, February 1997.
- [3] M. Nystrom. *Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512*. RFC 4231, December 2005.
- [4] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, May 2010.
- [5] Y. Nir, A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, June 2018.
- [6] A. Langley, M. Hamburg, S. Turner. *Elliptic Curves for Security*. RFC 7748, January 2016.